# Technology Stack

| | |
|---|---|
| **Date** | 07-07-2026 |
| **Team ID** | |
| **Project Name** | Smart Agricultural Production Optimization Engine (OptiCrop) |
| **Maximum Marks** | 2 Marks |

### Define Your Technology Stack:

Identify the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs used to build the solution. A well-chosen technology stack ensures that the application is scalable, maintainable, and aligned with the project requirements.

Below is a detail of the specific technologies chosen for each component of the architecture, with a brief justification for each choice.

### Technology Stack Details

| S.No | Architecture Component / Layer | Technology Chosen | Justification / Purpose |
|---|---|---|---|
| 1 | **Frontend / Client-Side** | HTML5, CSS3 (custom-styled, no framework) | A lightweight, dependency-free frontend keeps the app simple to deploy and maintain. Custom CSS (dark soil-green theme with Fraunces/Space Grotesk typography) gives a distinctive look without the overhead of a JS framework, since the app is a straightforward multi-page form-and-result flow with no complex client-side state. |
| 2 | **Backend / Server-Side** | Python + Flask | Python is the natural choice since the ML model (Scikit-learn) is also Python-based, avoiding any cross-language integration work. Flask is lightweight and sufficient for this project's scale — a handful of routes (Home, About, Find Your Crop, Predict) with no need for the added complexity of a larger framework like Django or FastAPI. |
| 3 | **Database / Data Storage** | Static CSV file (Crop_recommendation.csv) + Pickle (.pkl) files | The dataset (2,200 rows) is small and unchanging, so a CSV file is sufficient — no database server is needed for this scale. The trained model and scaler are serialized with Pickle and loaded directly into memory at app startup, avoiding the overhead of a database or external storage service. |
| 4 | **Cloud / Hosting / Deployment** | Render (free tier) | Render offers simple, direct deployment from a GitHub repository with no infrastructure to manage — well suited to a project of this scale. It automatically installs dependencies from requirements.txt and runs the app via gunicorn. The free tier does spin down after inactivity, |

| | | | |
|---|---|---|---|
| | | | causing a 30-50 second cold-start delay, which is an accepted tradeoff for a no-cost hosting solution. |
| 5 | **Version Control & CI/CD** | Git + GitHub | Git and GitHub provide version history and a public repository link for the project submission. No CI/CD pipeline (e.g. GitHub Actions) is currently set up, since deployments are manual pushes to Render — appropriate for a single-developer project at this scale, though automated testing/deployment would be a reasonable next step if the project grew. |
| 6 | **Third-Party APIs / Other Tools** | None currently integrated | The current version does not call any external APIs (no live weather, SMS, or maps services) — all predictions are based solely on the user's manually entered form values and the trained model. Integrating a live weather API (e.g. to auto-fill temperature/humidity/rainfall) is listed as a future enhancement, not part of the current build. |